

[Wiki »](#)

Лабораторная работа №1

В данной работе рассматривается первая задача, которая возникает перед разработчиком операционной системы: запуск на компьютере произвольной программы. Эта задача не так проста, как может показаться на первый взгляд. При разработке обычных программ способ их запуска, поддержка ввода-вывода и другие необходимые для работы условия считаются по умолчанию выполненными: программа имеет в качестве окружения операционную систему и стандартную библиотеку, что позволяет сконцентрироваться на реализации нужного алгоритма. При разработке операционной системы ситуация кардинально меняется: необходимо напрямую работать с аппаратным обеспечением компьютера, вручную реализовывать программную поддержку его возможностей и особенностей. В результате разработка даже, казалось бы, простейшей программы, выводящей на экран произвольную строку, требует глубокого понимания процесса загрузки компьютера, структуры физической памяти, аппаратной поддержки ввода-вывода и т.д.

Подготовка окружения

Для выполнения лабораторных работ необходимо запустить операционную систему JOS в виртуальной машине QEMU. Установка QEMU и использование виртуальной машины описаны в [соответствующей статье](#). После установки QEMU нужно клонировать репозиторий с исходным кодом лабораторных работ и переключиться на ветку lab1.

```
$ git clone http://sed.ispras.ru/git/oscourse-2022-mipt -b lab1 oscourse
```

Затем нужно перейти в каталог с репозиторием и собрать JOS, выполнив make. После этого можно запустить ее, выполнив make qemu:

```
$ cd oscourse
$ make
$ make qemu
```

Если всё сделано правильно, в окне QEMU появится текст следующего вида:

```
BdsDxe: failed to load Boot0001 "UEFI QEMU DVD-ROM QM00003 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Secondary,Master,0x0): Not Found
BdsDxe: loading Boot0002 "UEFI QEMU HARDDISK QM00001 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Primary,Master,0x0)
BdsDxe: starting Boot0002 "UEFI QEMU HARDDISK QM00001 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Primary,Master,0x0)
JOS: Awaiting debugger connection
```

Данные строки означают, что виртуальная машина QEMU запустила прошивку, совместимую с UEFI BIOS, и передала управление загрузчику операционной системы JOS. В текущем виде загрузчик операционной системы ожидает подключения отладчика для продолжения загрузки скелета ядра JOS.

Загрузка платформ на базе x86

Первые компьютеры, основанные на 16-битных процессорах Intel 8088, могли адресовать только 1 МБ физической памяти. Их физическое адресное пространство начиналось с 0x00000000, но заканчивалось на 0x000FFFFF вместо 0xFFFFFFFF. Эти компьютеры могли использовать только область размером в 640 КБ, часто называемую «нижняя область памяти» (Low Memory), причем некоторые из них были сконфигурированы для использования только 16, 32 или 64 КБ памяти. Область размером 384 КБ от 0x000a0000 до 0x000FFFFF была зарезервирована для специального применения, например, в качестве буфера дисплея или для отображения микропрограммы, находящиеся в энергонезависимой памяти. Наиболее важной частью этой зарезервированной области памяти являлся [BIOS](#) (базовая система ввода-вывода), который занимал область размером 64 КБ от 0x000F0000 до 0x000FFFFF. В ранних компьютерах BIOS находился в настоящих ПЗУ, доступных только для чтения, в современных компьютерах BIOS хранится в перезаписываемой флэш-памяти.

Современный BIOS, также называемый UEFI BIOS, архитектурно отличается от первоначального BIOS для компьютеров 8086 и обычно занимает больше памяти, порядка нескольких мегабайт. UEFI BIOS (или UEFI Firmware) является [переносимой спецификацией](#) начальной загрузки платформы, которая описывает интерфейсы взаимодействия между операционной системой и оборудованием. Теневая копия UEFI BIOS на x86 расположена в памяти по адресу 0xFFFFFFFF и ниже. Это связано в том числе с тем, что вектор начальной загрузки процессора (Reset Vector) находится по адресу 0xFFFFFFFF0. Существуют реализации UEFI BIOS не только для x86, но также для ARM и RISC-V. QEMU использует открытую реализацию UEFI BIOS под названием [OVMF](#). Для компьютеров, которые не поддерживают UEFI BIOS, можно использовать среду эмуляции UEFI [TianoCore DuetPkg](#), актуальная версия которой находится [здесь](#).

Любой BIOS отвечает за выполнение основных операций инициализации платформы, таких как активация видеокарты или проверка объема установленной памяти. После выполнения инициализации BIOS загружает операционную систему с носителя или из сети, а затем передает ей контроль над компьютером. В числе необходимой информации о платформе, которую должен предоставить операционной системе загрузчик, является информация об оперативной памяти: её общем количестве и назначении отдельных диапазонов. Современные x86-процессоры могут поддерживать более 4 ГБ физической памяти и использовать области выше 0xFFFFFFFF. Для информирования операционной системы о местоположении зарезервированных и иных областей, таких как конфигурационное пространство PCI, BIOS реализует интерфейсы по прерыванию 15h AX=E820h (Legacy BIOS) или по вызову GetMemoryMap (UEFI BIOS). Понять различные принципы работы UEFI BIOS можно в разделе [введение в UEFI](#). Прочтите его перед дальнейшим выполнением лабораторной работы. Подробнее о зарезервированных областях памяти и организации архитектуры PCI устройств можно прочитать [здесь](#) и [здесь](#).

Отладка загрузчика

В случае отсутствия иных загрузочных опций в соответствии со спецификацией UEFI BIOS ищет загрузчик на диске по фиксированному пути (Removable Media Path), который соответствует EFI\BOOT\BOOTX64.efi для Intel 64 и EFI\BOOT\BOOTIA32.efi для Intel 32. Так как загрузчики операционных систем грузятся прошивкой UEFI по случайному адресу и находятся в формате [Portable Executable](#), для подключения с помощью отладчика потребуются отдельные скрипты. Готовые скрипты для отладки как прошивки, так и загрузчика можно найти [здесь](#).

1. Откройте два окна терминала.
2. В первом окне терминала перейдите в каталог с исходным кодом лабораторных работ и запустите виртуальную машину отладки QEMU в режиме отладки, указав порт 8864 для подключения отладчика GDB:

```
GDBPORT=8864 make qemu-gdb
```

В результате выполнения команды виртуальная машина QEMU остановится сразу перед началом выполнения процессором первой инструкции и будет ожидать.

3. Во втором окне терминала клонируйте репозиторий с отладочными скриптами в любой удобный каталог и перейдите в подкаталог Debug:

```
git clone --depth=1 https://github.com/acidanthera/OpenCorePkg
cd OpenCorePkg/Debug
```

4. Произведите подключение к QEMU с помощью отладчика:

```
EFI_DEBUGGER=gdb EFI_ARCH=X64 ./efidebug.tool
```

Запустится отладчик GDB и появится текст следующего вида:

```
GNU gdb (GDB) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-unknown-linux-musl".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from GdbSyms/Bin/X64_GCC5/GdbSyms.debug...
The target architecture is set to "i386:x86-64:intel".
Remote debugging using localhost:8864
0x000000000000ffff in ?? ()
No EFI_SYSTEM_TABLE...
Function "DebugBreak" not defined.
Make breakpoint pending on future shared library load? (y or [n])
```

5. Так как прошивка ещё не была загружена, необходимо передать команду `c` (continue) и дождаться запуска загрузчика JOS, определить который можно по появлению строки `JOS: Awaiting debugger connection` в первом окне. После этого необходимо остановить отладчик комбинацией `Ctrl+C` и загрузить символы прошивки командой `reload-uefi`:

```
(gdb) c
Continuing.

Program received signal SIGINT, Interrupt.
0x000000001e6b1fff in ?? ()
(gdb) reload-uefi
EFI_SYSTEM_TABLE_POINTER @ 0x1f800000
EFI_SYSTEM_TABLE @ 0x1f9ee018
Connected to EDK II (Rev. 0x10000)
ConfigurationTable @ 0x1f9eec98, 0xa entries
DebugImageInfoTable @ 0x1f8e8018, 0x5f entries
Found 95 images...
Loading new symbols...
add symbol table from file "/home/jbes/src/cont/osc/LoaderPkg/UDK/Build/OvmfX64/RELEASE_GCC5/X64/MdeModulePkg/Core/Dxe/DxeMa
.text_addr = 0x1febe240
.data_addr = 0x1fed2e80
...множество подобных строк опущены...
(gdb) bt
#0 0x000000001e6b1fff in UefiMain (ImageHandle=0x1ee84e98, SystemTable=0x1f9ee018) at /home/jbes/src/cont/osc/LoaderPkg/UDK/
#1 0x000000001e6b29cb in ProcessModuleEntryPointList (ImageHandle=0x1ee84e98, SystemTable=0x1f9ee018) at /home/jbes/src/cont/osc/LoaderPkg/UDK/
#2 0x000000001e6b666f in _ModuleEntryPoint (ImageHandle=0x1ee84e98, SystemTable=0x1f9ee018) at /home/jbes/src/cont/osc/LoaderPkg/UDK/
#3 0x000000001fecf625 in CoreStartImage (ImageHandle=0x1ee84e98, ExitDataSize=0x1e7b6618, ExitData=0x1e7b6610) at /home/jbes/src/cont/osc/LoaderPkg/UDK/
#4 0x000000001f21d2e4 in EfiBootManagerBoot (BootOption=BootOption@entry=0x1e7b65c8) at /home/jbes/src/cont/osc/LoaderPkg/UDK/
#5 0x000000001f22016a in BootBootOptions (BootManagerMenu=0x1febdb40, BootOptionCount=5, BootOptions=0x1e7b6518) at /home/jbes/src/cont/osc/LoaderPkg/UDK/
#6 BdsEntry (This=<optimized out>) at /home/jbes/src/cont/osc/LoaderPkg/UDK/MdeModulePkg/Universal/BdsDxe/BdsEntry.c:1062
#7 0x000000001fec8535 in DxeMain (HobStart=0x1f8ea018) at /home/jbes/src/cont/osc/LoaderPkg/UDK/MdeModulePkg/Core/Dxe/DxeMain.c:1062
#8 0x000000001fec856c in ProcessModuleEntryPointList (HobStart=<optimized out>) at /home/jbes/src/cont/osc/LoaderPkg/UDK/BootManager/BdsEntry.c:1062
#9 _ModuleEntryPoint (HobStart=<optimized out>) at /home/jbes/src/cont/osc/LoaderPkg/UDK/MdeModulePkg/Library/DxeCoreEntryPoint/DxeCoreEntryPoint.c:1062
#10 0x000000001fec7ef in ?? ()
#11 0x0000000000000000 in ?? ()
(gdb)
```

6. С этого момента вы можете осуществлять пошаговое выполнение кода загрузчика JOS с помощью команд `n` (next) и `s` (step). Для выхода из бесконечного цикла ожидания отладчика используйте команду `p Connected = 1`. Данная команда обновит значение переменной и позволит продолжить выполнение процесса загрузки.

Перед продолжением убедитесь, что вы понимаете, почему выполнение кода останавливается в цикле `while (!Connected) { }`.

Устройство загрузчика JOS

Загрузчик JOS использует средство разработки программного обеспечения UEFI под названием [TianoCore EDK II](#). В системе сборки EDK II программное обеспечение группируется в пакеты (packages), которые состоят из драйверов и приложений. Подробнее о системе сборки EDK II можно прочесть в [спецификации системы сборки EDK II](#) и на [TianoCore Wiki](#). EDK II пакет JOS находится в каталоге `LoaderPkg`, который состоит из одного UEFI приложения — загрузчика `Loader`, исходники которого находятся в каталоге `LoaderPkg/Loader`. Для упрощения процесса сборки клонирование EDK II и передача всех необходимых команд для компиляции загрузчика, а также создания необходимой структуры каталогов для работы QEMU, выполняется в рамках системы сборки `make`.

Пользовательской точкой входа в загрузчик является функция `UefiMain` в файле `Bootloader.c`. Её имя, как и файл, в котором она находится, определяется в манифесте сборки загрузчика — `Loader.inf`. В манифесте также указываются используемые в загрузчике библиотеки. Служебные библиотеки точки входа, например, `UefiApplicationEntryPoint`, осуществляют проверку совместимости платформы в момент запуска загрузчика, проводят инициализацию других библиотек через их конструкторы и передают управление в пользовательскую точку входа. В случае завершения работы приложения данная библиотека также вызывает деструкторы используемых библиотек. Вместо стандартной библиотеки языка C в EDK II используются собственные библиотечные функции и сервисы. О возможностях `UEFI Boot Services (gBS)` и `UEFI Runtime Services (gRT)` можно прочесть в спецификации `UEFI`. О возможностях используемых библиотек можно узнать в соответствующих заголовочных файлах пакета, интерфейс которого конкретная библиотека реализует. Например, интерфейс библиотеки отладки, который предоставляет средства вывода отладочных сообщений и встроенные проверки (assertions), описан в `LoaderPkg/UDK/MdePkg/Include/Library/DebugLib.h`.

В процессе работы загрузчик выполняет следующие шаги:

1. Ожидает подключения отладчика GDB (если данный код активен).
2. Получает актуальную дату из `UEFI Runtime Services` и выводит её в консоль.
3. Выделяет память для аргументов загрузки ядра JOS.
4. Инициализирует параметры графики.
5. Загружает ядро JOS из файловой системы по пути `EFI\BOOT\kernel` и отображает в память.
6. Настраивает адресное пространство для работы ядра и завершает выполнение `UEFI Boot Services`.
7. Выполняет переход к точке входа ядра JOS.

Изучите код загрузчика и убедитесь, что вы понимаете логику работы кода загрузчика на всех этапах. Обратите внимание, что стиль написания кода загрузчика JOS отличается от остального стиля кода в JOS и близок к стилю кода EDK II. Спецификация стиля написания кода для EDK II доступна на [Wiki TianoCore](#). Описание стиля написания кода остальной части JOS доступно в файле `CODING` и может быть применено с помощью [ClangFormat](#) посредством команды `make format`.

Загрузка ядра

Рассмотрим более подробно пятый этап работы загрузчика, реализованный в функции `LoadKernel`. JOS использует обычный механизм загрузки с жесткого диска. Загрузчик получает содержимое ядра JOS посредством использования протокола `Simple FileSystem`. Данный протокол описан в спецификации `UEFI` и позволяет скрыть внутреннее устройство драйвера носителя. Так как разработка драйверов устройств является очень важной частью разработки ОС, более подробно к ней мы вернёмся в следующих частях лабораторной работы. Альтернативным способом загрузки является использование протокола `Load Image`, который позволяет реализовать сетевую загрузку.

Ядро находится на диске в двоичном формате `ELF (Executable and Linkable Format, «Исполняемый и компоуемый формат»)`. При компиляции и компоновке программы на C (например, ядра JOS) компилятор создает на основе каждого исходного файла (.c) объектный (.o), содержащий инструкции процессора в двоичном формате. Затем компоновщик объединяет все скомпилированные объектные файлы в единый двоичный образ, который в случае с JOS является двоичным файлом в формате `ELF` и находится в `obj/kern/kernel`.

`ELF`-файл можно рассматривать как заголовок с метаинформацией, за которой идет несколько секций программы, каждая из которых представляет собой непрерывный кусок кода или данных, предназначенный для загрузки в память по указанному адресу. Загрузчик никак не меняет код или данные, он загружает их в память и начинает выполнение.

Метаинформация в `ELF`-файлах состоит из `ELF`-заголовка фиксированной длины и заголовка программы переменной длины, в котором перечислены все секции, которые должны быть загружены. Определения на языке C для этих заголовков, позволяющие программно обрабатывать их, находятся в `LoaderPkg/Include/Elf64.h`. Нас интересуют следующие секции программы:

```
.text: исполняемые инструкции программы;
.rodata: данные только для чтения, например, строковые константы (мы не будем запрещать их чтение на аппаратном уровне);
.data: инициализированные данные программы, например, глобальные переменные, объявленные с помощью конструкций вида int x = 5;
```

Когда компоновщик вычисляет расположение программы в памяти, он оставляет пространство для неинициализированных глобальных переменных, объявленных с помощью конструкций вида `int x`, в секции `.bss`, которая следует сразу после `.data`. Стандарт языка C требует, чтобы неинициализированные глобальные переменные инициализировались нулевым значением. Таким образом, нет необходимости хранить содержимое `.bss` в `ELF`-файле. Вместо этого компоновщик записывает только адрес и размер `.bss`. Загрузчик или сама программа должны инициализировать секцию `.bss` нулевыми значениями.

Изучите полный список названий, размеров и адресов всех секций в ядре с помощью `objdump (x86_64-jetos-elf-objdump для macOS)`:

```
$ objdump -h obj/kern/kernel
```

Вы увидите значительно больше секций. Большая часть секций содержит отладочную информацию, которая обычно включается в исполняемый файл программы, но не загружается в память.

Обратите особое внимание на значения `VMA (адреса связывания)` и `LMA (адреса загрузки)` секций `.bootstrap` и `.text`. Адрес загрузки — это тот адрес, по которому секция должна быть загружена в память. В структуре, описывающей формат `ELF`, это значение хранится в поле `ph->r_p_a` (в данном случае это действительно физический адрес, однако в спецификации `ELF` фактическое значение этого поля является неопределённым). Адрес связывания секции — это адрес, с которого секция ожидает начать выполнение. Компоновщик использует адрес связывания, например, при вычислении адреса глобальной переменной, в результате чего исполняемый файл, как правило, не будет работать, если он начинает выполнение с неправильного адреса. Существует также возможность генерации позиционно-независимого кода, который не содержит таких абсолютных адресов. Эта возможность широко используется в современных библиотеках, но она связана с затратами производительности и увеличением сложности, поэтому мы не будем её использовать.

В секции `.bootstrap` содержится код начальной загрузки ядра JOS, осуществляющий первичную настройку виртуального адресного пространства ядра, которое является обязательным в 64-битном режиме работы процессора Intel. Так как окружение `UEFI` работает в режиме однозначного отображения (`Identity Mapping`), т.е. отображения, в котором физический адрес равен виртуальному, ядро JOS на ранней стадии запуска переходит к другому отображению, исполнение в котором производится с высоких адресов. По этой причине `VMA` и `LMA` адреса секций `.text` и других не совпадают. Подробнее работу с виртуальной памятью мы рассмотрим в следующих лабораторных работах.

Для удобства реализации секции `ELF` файла группируются в соответствии с их правами доступа в сегменты, и именно по ним в соответствии со спецификацией должна осуществляться загрузка образа в память. В текущей версии JOS для упрощения демонстрации отсутствует защита памяти данных от исполнения. В других операционных системах является стандартной практика [W^X](#) — запрета исполнения изменяемых данных. Посмотреть список сегментов можно также с помощью команды `objdump`:

```
$ objdump -p obj/kern/kernel
```

Помимо информации о секциях и сегментах, есть ещё одно важное поле в заголовке ELF: `e_entry`. В этом поле находится адрес точки входа в программу — адрес в одной из секций, с которого программа должна начать выполнение. Для ядра JOS этот адрес находится в секции `.bootstrap` и равен функции `_head64`. Вы также можете получить адрес точки входа с помощью команды `objdump`:

```
$ objdump -f obj/kern/kernel
```

Таким образом должно быть понятно, что загрузчик JOS читает ядро с диска по сегментам в память по адресу, соответствующему адресу связывания секции, а затем переходит к точке входа в ядро.

Доработка загрузчика

В качестве знакомства с кодом загрузчика JOS внесите следующие изменения в код:

1. Измените разрешение экрана в загрузчике на такое, с которым бы вам было наиболее удобно работать.
2. Отключите остановку выполнения для подключения отладчика в загрузчике.

Подсказка: в UEFI вывод на экран осуществляется с помощью протокола Graphics Output, устройство которого описано в спецификации UEFI. Для изменения разрешения вы можете внести изменения в функцию `InitGraphics`, обойдя все доступные разрешения и выбрав предпочтительное посредством функций `QueryMode` и `SetMode` протокола. Заголовочный файл протокола также находится по пути `LoaderPkg/UDK/MdePkg/Include/Protocol/GraphicsOutput.h`.

Отладка ядра

Если в предыдущих пунктах всё было сделано правильно, то при выполнении команды `make qemu` в окне QEMU появится текст следующего вида:

```
BdsDxe: failed to load Boot0001 "UEFI QEMU DVD-ROM QM00003 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Secondary,Master,0x0): Not Found
BdsDxe: loading Boot0002 "UEFI QEMU HARDDISK QM00001 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Primary,Master,0x0)
BdsDxe: starting Boot0002 "UEFI QEMU HARDDISK QM00001 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Primary,Master,0x0)
JOS: Loading at 2021-2-7 - 9:27:9...
JOS: Found ACPI 1.0 RSDP table 1FB7E000
JOS: Found ACPI 2.0 RSDP table 1FB7E014
JOS: CR0 - 80010033 CR3 - 1FC01000
JOS: Loading kernel image...
JOS: Loading debug tables...
JOS: MinAddress: 0x1500000, MaxAddress: 0x1619000, Kernel size: 0x119000
JOS: Loaded kernel with 1500000 entry point
JOS: LoaderParams pointer: 1F940018
JOS: KernelCallGate: 1E6D06B7
6828 decimal is XXX octal!
END: 0x8041619000
Framebuffer initialised
entering test_backtrace 5
entering test_backtrace 4
entering test_backtrace 3
entering test_backtrace 2
entering test_backtrace 1
entering test_backtrace 0
leaving test_backtrace 0
leaving test_backtrace 1
leaving test_backtrace 2
leaving test_backtrace 3
leaving test_backtrace 4
leaving test_backtrace 5
Welcome to the JOS kernel monitor!
Type 'help' for a list of commands.
K>
```

Всё, что выведено после строки "JOS: KernelCallGate: 1E6D06B7...", выведено скелетом ядра JOS. "K>" — приглашение монитора (интерактивной программы управления). Эти команды выводятся в окне QEMU и в терминале: для тестирования ядро JOS настроено таким образом, что оно выводит данные не только на экран, но и в последовательный порт, данные из которого QEMU использует как стандартный вывод. Аналогично, ядро JOS принимает данные как с клавиатуры, так и из параллельного порта, то есть можно вводить команды как в окне QEMU, так и в терминале. В данный момент существует только две команды: `help` и `kerninfo`.

По аналогии с отладкой загрузчика, откройте два окна терминала, и в обоих перейдите в каталог с исходным кодом лабораторных работ. В одном выполните команду `make qemu-gdb`. Во втором выполните команду `gdb` (или `ggdb` на macOS). Запустится отладчик GDB и появится текст следующего вида:

```
GNU gdb (GDB) 9.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-apple-darwin19.4.0".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
+ target remote localhost:25501
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x000000000000ffff in ?? ()
```

```
+ symbol-file obj/kern/kernel
(gdb)
```

Вы можете установить адрес точки прерывания в GDB командой `b`, для установки точки прерывания на адрес может быть использован символ `*`. Например, `b * 0x7c00` устанавливает точку прерывания на адрес `0x7C00`, а `b func` — на символ `func`. Как только точка срабатывает, вы можете продолжить выполнение с помощью команд `c` и `si`: `c` заставляет QEMU продолжить выполнение до следующей точки прерывания (или до нажатия Ctrl-C в GDB), `si N` позволяет выполнить сразу `N` инструкций подряд.

Чтобы изучить инструкции в памяти (помимо следующей, которая будет выполнена — её GDB выводит автоматически), можно использовать команду `x/i`. Эта команда имеет синтаксис `x/Ni ADDR`, где `N` — количество последовательных инструкций, а `ADDR` — адрес, с которого нужно начать.

Для отладки ядра с первой инструкции установите точку останова на функцию `_head64` и продолжите выполнение:

```
(gdb) b _head64
Breakpoint 1 at 0x150000: file kern/bootstrap.S, line 21.
(gdb) c
Continuing.

Breakpoint 1, _head64 () at kern/bootstrap.S:21
21     cli
(gdb)
```

Сравните код в отладчике с файлом `obj/kern/kernel.asm`. Этот файл представляет собой дизассемблированный код ядра, который создается после компиляции. Он позволяет увидеть, где именно находится тот или иной код ядра, и упрощает отслеживание с помощью GDB. Описание ассемблера x86 можно найти в [руководстве Intel по архитектуре процессора](#). Краткое введение в 64-битный ассемблер описано в отдельной [статье](#). Логика работы ассемблера x86 без 64-битного режима описано в книге [PC Assembly](#).

Для удобного просмотра исходного кода также можно воспользоваться текстовым интерфейсом TUI. Переключение в TUI можно выполнить командой `layout next`. Обратите внимание, что в разных видах TUI доступны разные окна, которые позволяют одновременно просматривать отлаживаемую программу из разных перспектив: ассемблера, исходного кода, значений регистров. Для выбора активного окна в TUI используйте команду `focus`, например, `focus cmd` для выбора окна командной строки. Для выхода из TUI используйте команду `tui disable`.

Последовательно выполняйте инструкции ядра из файлов `kern/bootstrap.S` и `kern/entry.S` вплоть до функции `i386_init()`, находящейся в `kern/init.c`. При необходимости сравнивайте исходный код ядра с его дизассемблированным кодом (`obj/kernel/kernel.asm`) и выводом GDB. Также может быть полезным просмотр содержимого регистров с помощью GDB.

Для просмотра содержимого регистров процессора воспользуйтесь командой `info registers`. Эта команда GDB выводит содержимое всех регистров CPU. Для печати содержимого отдельного регистра, например, RAX, нужно указать регистр как аргумент команды: `info registers rax`. Альтернативным способом инспекции значений, в том числе регистров, является команда `p`, например, `p $rax`.

Исследовать память можно с помощью команды GDB `x`. В руководстве GDB есть полная информация, но сейчас достаточно знать, что команда `x/Nx ADDR` выводит `N` слов по адресу `ADDR`. (Обратите внимание, что оба "x" в команде находятся в нижнем регистре.) Внимание: размер слова не является универсальным стандартом. В ассемблере GNU размер слова — 2 байта (буква `w` в команде `xorw` означает слово и соответствует 2 байтам)

После изучения кода начальной загрузки ядра ответьте на следующие вопросы:

- Как передаются параметры UEFI загрузчика в ядро, и куда они сохраняются?
- Какой стек использует C-код ядра, и где он выделяется?
- В какой момент код начинает выполняться по "верхним" адресам?
- Какие значения CR3 и GDT используются на протяжении работы ядра?
- Почему в стартовом коде не требуется обновлять GDT в отличие от CR3?

Форматированный вывод

Для вывода данных монитор использует функции форматированного вывода (в частности, `printf`). Обычно для этого используются функции из стандартной библиотеки C, однако в данном случае она избыточна. Поэтому JOS использует собственный упрощенный вариант этих функций.

Просмотрите файлы `kern/printf.c`, `lib/printfmt.c` и `kern/console.c`. В одном из них пропущен небольшой фрагмент кода: печать чисел в восьмеричном виде при использовании последовательности `%o`. Найдите это место и восполните недостающий фрагмент.

Проверка и выкладка лабораторной работы

По окончании выполнения каждой работы необходимо убедиться, что её код компилируется всеми поддерживаемыми компиляторами во всех поддерживаемых конфигурациях и корректно выполняется. Если ранее вы использовали компилятор GCC, убедитесь, что ваша лабораторная корректно работает и с компилятором LLVM, используя команду `JOSLLVM=1 make qemu`. В противном случае проверьте GCC, указав `JOSLLVM=0`. В следующих лабораторных работах у вас появятся новые переменные окружения, позволяющие определить другие конфигурации запуска JOS, а также команда `make grade`, которая позволяет выполнить автоматические тесты для текущей лабораторной работы.

По окончании выполнения работы следует создать в репозитории новую ветку `working-lab1` и сохранить в ней внесенные изменения. После этого нужно добавить удаленный репозиторий для сохранения результатов по предоставленному преподавателем адресу и отправить в него эту ветку, например:

```
git checkout -b working-lab1
git add <...>
git commit -m "Lab 1 done."
git remote add myrepo http://sed.ispras.ru/git/<name>-osprc # здесь <name> - ваш логин на sed.ispras.ru
git push myrepo working-lab1
```